



Laboratório Técnico 04: O Transformer Completo "From Scratch"

Nas últimas semanas, vocês atuaram como engenheiros dissecando os motores do Transformer. No Lab 01, construíram o motor matemático do *Scaled Dot-Product Attention*. No Lab 02 e 03, focaram no *Causal Masking*, na ponte *Cross-Attention* e mapearam o fluxo do *Encoder* com suas conexões residuais (*Add & Norm*) e redes densas (*FFN*).

Agora vocês vão instanciar todos esses códigos legados e construir a arquitetura completa (Encoder-Decoder) para realizar um teste de tradução fim-a-fim de uma frase de brinquedo (*toy sequence*).

Objetivos de Aprendizagem:

1. Aplicar engenharia de software para integrar módulos de redes neurais separados em uma única topologia coerente.
2. Garantir o fluxo correto de tensores passando pelas camadas *Add & Norm* e *Feed-Forward*.
3. Acoplar o *Loop* Auto-regressivo de inferência na saída do Decoder.

Tarefa 1: Refatoração e Integração (Os Blocos de Montar)

O que deve ser codificado: Importe ou reescreva as funções criadas nos laboratórios anteriores, garantindo que elas aceitem tensores dinâmicos de PyTorch ou NumPy. Você precisará ter instanciado:

1. **O Mecanismo de Atenção:** A sua função genérica `scaled_dot_product_attention(Q, K, V, mask)`.
2. **A Rede FFN:** O *Position-wise Feed-Forward Network* com uma expansão de dimensão (ex: de 512 para 2048) e ativação ReLU no meio.
3. **Add & Norm:** A lógica da Conexão Residual somada à Normalização de Camada, dada pela equação $\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$.

Tarefa 2: Montando a Pilha do Encoder

Fundamento: O Encoder não apenas lê a entrada, ele a contextualiza bidirecionalmente. **O que deve ser codificado:**

1. Crie uma função ou classe `EncoderBlock(x)`.
2. O fluxo exato para o tensor de entrada simulado X (já somado com o *Positional Encoding*) deve ser:





- Passar pelo **Self-Attention** (gerando as matrizes Q, K, V a partir de X).
 - Aplicar **Add & Norm**.
 - Passar pelo **FFN**.
 - Aplicar **Add & Norm** novamente.
3. Faça esse bloco ser empilhável para processar a matriz de memória rica (Z).

Tarefa 3: Montando a Pilha do Decoder

Fundamento: O Decoder precisa gerar a resposta baseando-se no que já gerou e na memória Z do Encoder, sem trapacear. **O que deve ser codificado:**

1. Crie uma função ou classe DecoderBlock(y, Z).
2. O fluxo exato do tensor alvo simulado Y deve ser:
 - Passar pelo **Masked Self-Attention** (usando a máscara causal do Lab 2 para zerar o futuro com -infinito).
 - Aplicar **Add & Norm**.
 - Passar pela ponte de **Cross-Attention** (onde Q vem da saída anterior, e K, V vêm da matriz Z do Encoder).
 - Aplicar **Add & Norm**.
 - Passar pelo **FFN** e pelo último **Add & Norm**.
3. Ao final, passe o resultado por uma camada Linear que projeta as dimensões para o tamanho do vocabulário e aplique Softmax para obter as probabilidades.

Tarefa 4: A Prova Final (Inferência)

O que deve ser codificado:

1. Instancie o modelo completo usando tensores fictícios.
2. Passe um tensor de entrada encoder_input simulando a frase "*Thinking Machines*".
3. Implemente o laço auto-regressivo while integrado com o modelo:
 - O Decoder deve iniciar com o token <START>.
 - A cada iteração, ele prevê a próxima palavra baseada nas probabilidades do Softmax.
 - A nova palavra é concatenada à entrada do Decoder, reiniciando o ciclo até a geração do token <EOS>.

Instruções de Entrega e Contrato Pedagógico:

- **Repositório Git:** A entrega será exclusivamente via Git.





- **Versionamento:** O *commit* avaliado deverá possuir a *tag* ou *release* "v1.0".
- **Prazo e Penalidades:** O código submetido tem data limite rígida (até às 23h59). Atrasos sofrem penalidade direta: 1 dia (-20%), 2 a 3 dias (-50%), acima de 3 dias resulta em Nota 0.
- **Uso de Inteligência Artificial:** É permitido o uso de IA para *brainstorming* ou geração de *templates* básicos, **porém**, a lógica matemática da montagem do modelo deve ser documentada por você. É **obrigatória** a inclusão de uma nota no arquivo README.md: "*Partes geradas/complementadas com IA, revisadas por [Seu Nome]*". Submeter códigos gerados integralmente pela IA sem o devido entendimento e citação é configurado como plágio e resultará em nota 0.

